



Daily Task

Task 1: Sentiment-with-CoT from Customer Reviews

Given a customer review scraped from an online e-commerce website, use an LLM to predict the sentiment (``positive`` / ``neutral`` / ``negative``) and **justify** the decision by explaining the reasoning step by step using **Chain-of-Thought** prompting.

1. Dataset

Scrape 5 - 10 customer reviews from any e-commerce platform (e.g., Amazon product reviews, restaurant reviews).

2. Prompt design : Design a few-shot Chain-of-Thought prompt that:

- Shows examples of reviews with:
 - ``positive`` label + detailed reasoning,
 - ``neutral`` label + reasoning,
 - ``negative`` label + reasoning.
- States explicit steps such as:
 - “Identify all positive-sentiment phrases.”
 - “Identify all negative-sentiment phrases.”
 - “Check for contradictions or mixed sentiment.”
 - “Decide the final label and explain why.”

Task 2: Choosing right parameters using Tree-of-Thought

Perform hyperparameter-tuning on any ML algorithm (e.g., ``XGBoost``, ``RandomForest``, or ``LogisticRegression``) and use an LLM, guided by **Tree-of-Thought**, to analyze and explain why certain hyperparameter configurations look promising.



1. ML model and hyperparameter space

- Train **any ML model** (classification or regression) on a standard dataset (e.g., `sklearn.datasets`, Kaggle dataset, or your own data).

- Define a **small but meaningful hyperparameter grid** (e.g., 10–30 configurations).
- Perform **cross-validation** and store:
 - CV score (e.g., accuracy, F1, RMSE),
 - training time,
 - overfitting indicators (e.g., train vs. val gap).

2. Tree-of-Thought prompt design

- Guides the LLM to:
 - Group configurations by broadly promising ranges.
 - Evaluate bias-variance tradeoffs for each candidate.
 - Compare training time and performance.
 - Eventually pick **one “best” configuration** and justify it.